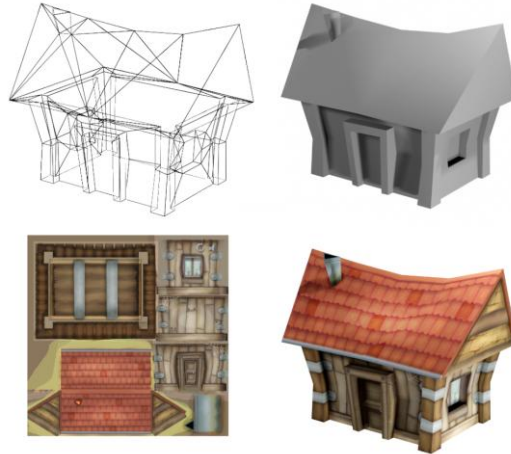




TEXTUREN

Bilder: Wikipedia

Texturen

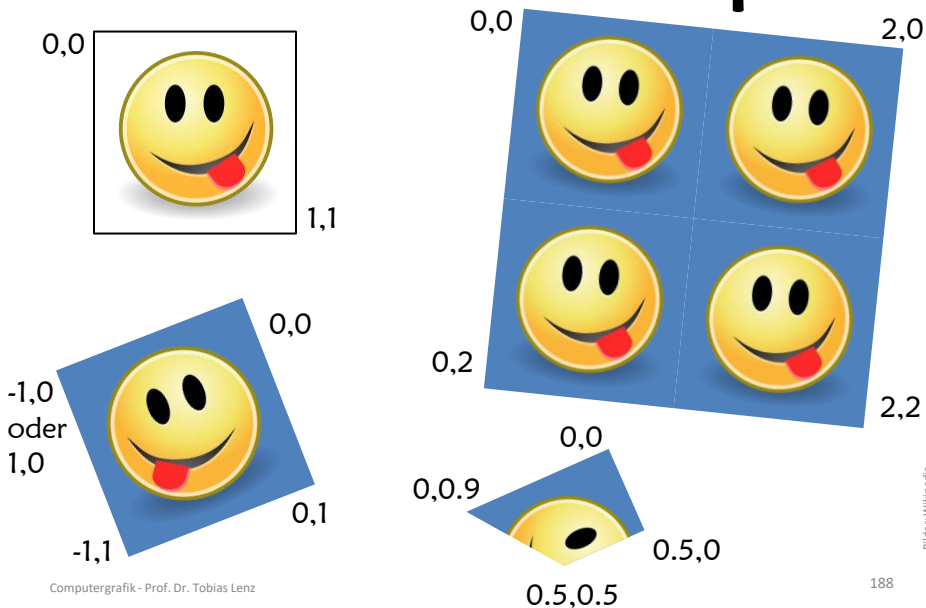
- Details nicht modellieren: aufmalen
 - Geometrie ist teuer, Textur ist billig
- 2D-Grafiken (Bilder, Fotos, ...) auf 3D-Objekte projizieren
- Begriff steht heute für alles in 2D mit Pixeln
 - Z-Buffer
 - Renderergebnis
 - ... siehe restliches Kapitel

Texturkoordinaten

- Textur ist Rechteck mit Koordinatenbereich 0,0 – 1,1
 - uv-Koordinaten
 - Auflösungsunabhängig
- Polygonecken Texturkoordinaten zuordnen
 - Ecke hat einmal uv (Duplizieren für mehrere)
 - Über Fläche interpolieren
 - Negative Koordinaten und größer 1 erlaubt
 - Führt zu Musterwiederholung / Spiegelung

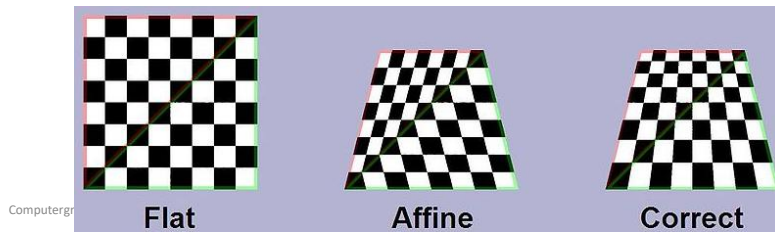


Texturkoordinaten: Beispiele



Perspektivische Korrektur

- Lineare Interpolation der uv-Koordinaten in Bildschirmraum ist perspektivisch falsch
- Über u/z und v/z (und $1/z$) interpolieren
 - $u = (u/z) / (1/z)$, $v = (v/z) / (1/z)$
 - Früher: Nur alle x Pixel korrekt berechnen
 - Heute: Vollständig korrekt in Hardware



Bilder: Wikipedia

Texturkompression

- Kenne ich schon: ~~NO is!~~ JPEG, nächste Folie
- Der große Speicherverbrauch moderner 3D-Welten stammt von Texturen
- Unkomprimiert möglich, aber hoher Speicherverbrauch **und langsamer**
- Komprimiert schneller, da weniger Bytes gelesen werden müssen
- Dekompression übernimmt Hardware

Texturkompression

- Sehr verbreitet: S3 Texture Compression
- Formate DXT1 bis DXT5
 - Unterschiedliche Behandlung des Alphakanals
- Lossy
- Feste Datenrate, fester Kompressionsfaktor
- Ideal für On-the-fly-Dekompression
- Probleme: Lizenzunklarheiten, für moderne Tricks nicht ideal
- Herstellerspezifische Modifikationen

DXT1-Kompression (kein Alpha)

- 4x4 Pixel mit je 24 Bit im Original
- Gespeichert werden zwei 16-Bit RGB-Farben (5 Bit für R und B, 6 Bit für G) c_0, c_1
- Zwei weitere Farben c_2, c_3 berechnen als gewichtetes Mittel aus c_0, c_1
- Für die 16 Originalpixel je eine 2-Bit Zahl i
- Als Farbe wird jeweils c_i verwendet
- Lossy: In jedem 4x4 Block gibt es nur 4 verschiedene Farben (statt 16)
- Speicher: statt $16 \cdot 24$ Bit, nur $2 \cdot 16 + 16 \cdot 2$ Bit

Beispiel

- Wie bei JPEG sind Artefakte zu finden
- Links: Original
- Rechts: DXT1
- Auch wie JPEG:
 - Artefakte auf scharfen Kanten sehr deutlich



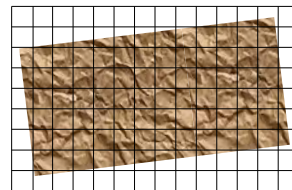
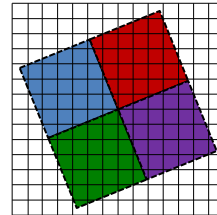
Bilder: Shawn Hargreaves, <https://shawnhargreaves.com/blog/texture-compression.html>

Texturkompression in OpenGL

- (Fast) kein Unterschied
- Programmierer lädt die Pixeldaten (z.B. 3 Byte pro RGB-Pixel) in ein Array
- Zur Grafikkarte schicken: `glTexImage2D(...)`
- Alternativ lädt man die komprimierten Daten in das Array
- Zur Grafikkarte schicken mit `glCompressedTexImage2D(...)`

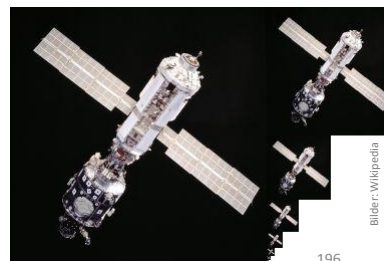
Sampling-Dichte

- *Undersampling*
 - Ein Texel für viele Pixel
 - Schlechte Qualität
 - „blockig“ oder „verwaschen“
- *Oversampling*
 - Viele Texel für ein Pixel
 - Flackern dünner Details
 - Alternativ: Hoher Aufwand
 - Gewichtung über viele Texel



MIP mapping

- MIP = *multum in parvo*
- Texturen werden abstandsabhängig sehr stark vergrößert/verkleinert
- Idee: Für entfernte Objekte kleinere Versionen der Texturen verwenden, für nahe Objekte größere
 - Beliebig viele Stufen

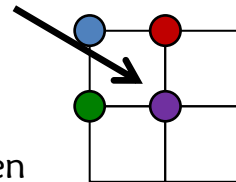


MIP mapping

- Größenstufen können einmalig (z.B. beim Laden), automatisch und mit aufwändigen Skalierungsverfahren generiert werden
- Welche Größe ist auszuwählen?
 - (Fast) alle Texel sollten im Bild zu sehen sein
- Höhere Geschwindigkeit bei gleicher/besserer Bildqualität
 - Kostet 1/3 mehr Speicher

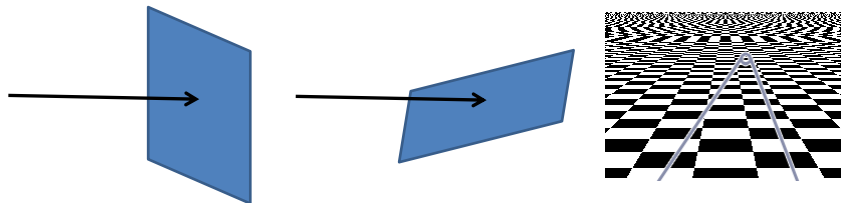
Texturinterpolation

- Pixelkoordinaten ganzzahlig, uv-Koordinaten rational
 - Texel werden nicht exakt getroffen
- Mögliche Färbung des Zielpixels:
 - Nächstliegenden Texel wählen
 - Gewichtete Mischung der 4 angrenzenden Texel (bilineare Filterung)
 - Lineare Interpolation zwischen den Werten zweier bilinear gefilterter MIP-Maps (trilineare Filterung)



Problem: Perspektive

- Bisherige Verfahren optimal wenn Oberfläche quadratisch und senkrecht zur Kamera
- Oberflächen mit starker Neigung zur Kamera sind nicht gleichmäßig in x- und y-Richtung verzerrt



Computergrafik - Prof. Dr. Tobias Lenz

199

Anisotroper Filter

- Ungleichförmig skalierte Texturen vorbereiten
- Oberfläche sollte achsenparallel sein
 - Funktioniert sehr gut für Boden/Wand
- Mehr vorbereitete Stufen
 - = größerer Winkelbereich
 - Heute bis zu 16 Stufen
 - Kaum von 4 zu unterscheiden



Computergrafik - Prof. Dr. Tobias Lenz

200

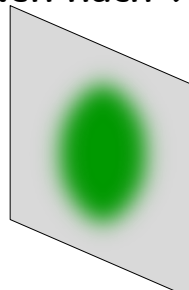
Texturinterpolation: Vergleich



Bilder: geforce.com

Transparenz

- Texturen haben Alpha-Kanal
 - „Deckung“ (*opaque*) 0 bis 1 wie rot, grün, blau
- Art der Verknüpfung wählbar (*blending*)
 - PMA: $\text{Texturfarbe} + (1 - \alpha) * \text{Zielpixel}$
- Zeichenreihenfolge von hinten nach vorne
- Tiefenpuffer deaktivieren

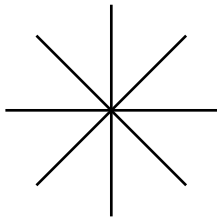


Texturen für alles

- Bisher: Textur beeinflusst die Objektfarbe
- Warum nicht auch...
 - Licht?
 - Schatten?
 - Reflexion?
 - Geometrie?
- Häufig nur Illusion!
- MIP mapping / Filtern lässt sich ebenso verwenden
- Kombination verschiedener Verfahren möglich
- (Mehrdimensionale) Prozedurale Texturen

Billboarding

- Flache Plakatwände mit Texturen aufstellen
- Senkrecht zur Kamera
- Sieht von allen Seiten identisch aus
- Mehrere Flächen

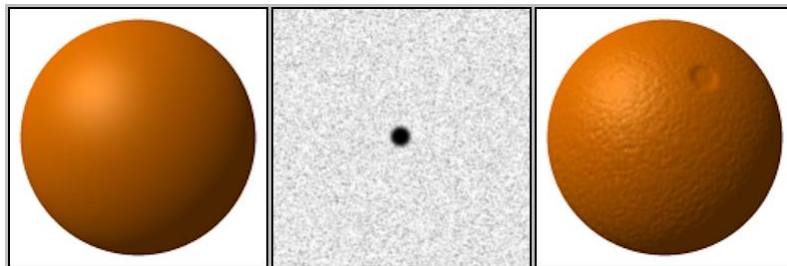


Billboardingbeispiel



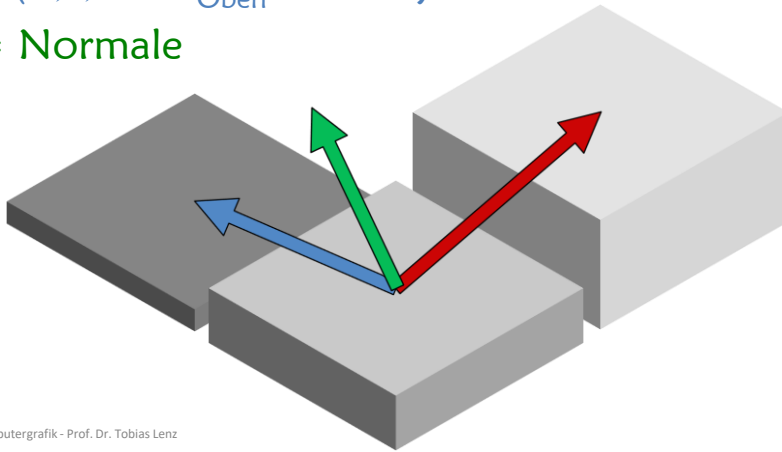
Bump mapping

- Leichte Höhenunterschiede der Oberfläche nicht modellieren, sondern „malen“
- Aus „Höhenkarte“ Steigung approximieren
- Normale entsprechend manipulieren



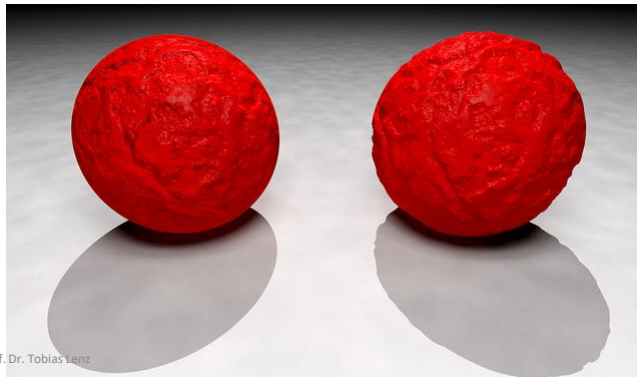
Bump mapping: Wie?

$$\begin{aligned} & (1, 0, \text{Höhe}_{\text{Rechts}} - \text{Höhe}) \\ & \times (0, 1, \text{Höhe}_{\text{Oben}} - \text{Höhe}) \\ & = \text{Normale} \end{aligned}$$



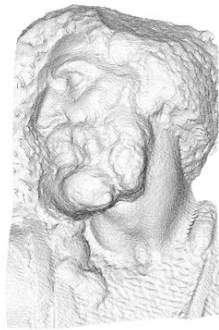
Bump mapping: Einschränkungen

- Oberfläche bleibt glatt (Silhouette, Schatten)
- Tiefeneindruck nicht blickwinkelabhängig
- Bumps werfen keine Schatten auf sich selbst

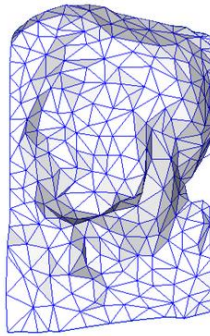


Normal mapping

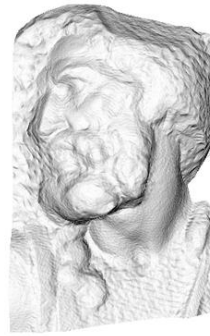
- Statt Höhenkarte direkt Normalen „malen“
- Automatisch aus Modelldetails berechnen



original mesh
4M triangles



simplified mesh
500 triangles



simplified mesh
and normal mapping
500 triangles

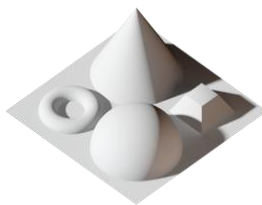
Computergrafik - Prof. Dr. Tobias Lenz

209

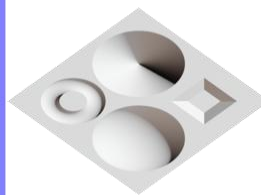
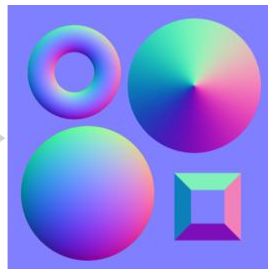
Bilder: Wikipedia

Normal mapping

- Normalenvektor (x,y,z) wird kodiert in Farbwert (r,g,b)
- Vergleich bump mapping:
 - + Bessere Kontrolle, weniger Rechenaufwand
 - Speicherverbrauch höher



Computergrafik - Prof. Dr. Tobias Lenz



210

Bilder: Wikipedia

Normal mapping: Wie?

- Normale hat Länge 1, also x,y,z von -1 bis 1
- Texturfarbwert (r,g,b) wird zu Vektor $(x,y,z) = (2r - 1, 2g - 1, 2b - 1)$
- Normalen in normal map gelten bei Kamera senkrecht vor Oberfläche
 - $(0, 0, 1)$ zeigt direkt zur Kamera
- Normalen der normal map so drehen, dass $(0,0,1)$ auf Oberflächennormale liegt

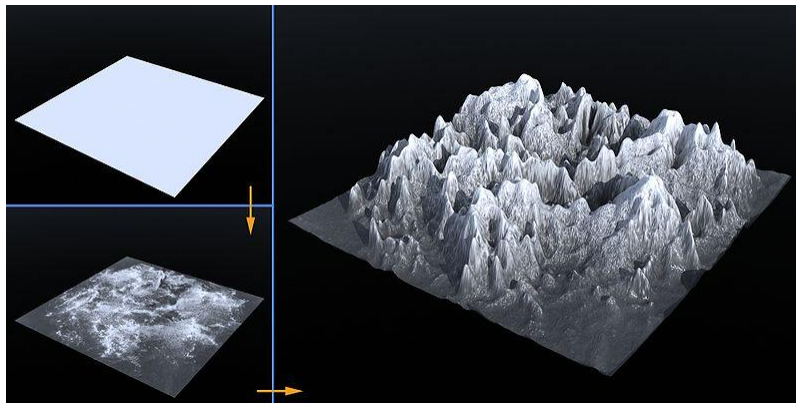
Normalen drehen

- Normal map hat Basis $\begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$
- Zielbasis im Bildschirmraum
 - U = Vektor entlang der U-Texturkoordinate
 - V = Vektor entlang der V-Texturkoordinate
 - N = Normalenvektor der Oberfläche
- Basistransformation:
$$\begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = \begin{pmatrix} U_x & V_x & N_x \\ U_y & V_y & N_y \\ U_z & V_z & N_z \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$

Displacement mapping

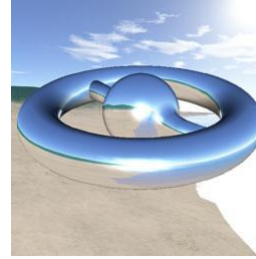
- Echte geometrische Verschiebung entsprechend einer Höhenkarte
 - Keine Illusion
 - Silhouette und Schatten korrekt
 - Objekt wirft Schatten auf sich selbst
- Erfordert zusätzliche Dreiecke
 - Mehr Rechenaufwand
 - In heutiger Hardware dynamisch möglich
- Geometrie kann auch vorberechnet werden, dann keine Textur nötig

Displacement mapping: Beispiel



Environment / Reflection mapping

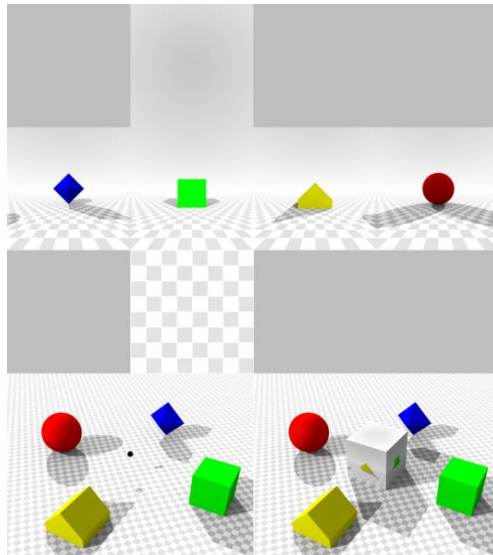
- „Schummelvariante“
 - Textur sieht aus wie Umgebung
- Korrekt
 - Objekt mit Kugel (sphere mapping) oder Würfel (cube mapping) umgeben
 - Welt aus Objektsicht auf Kugel/Würfel projizieren
 - Strahl zur Kamera an Normale reflektieren und mit Kugel/Würfel schneiden = Texel
- Problem: Objekt spiegelt sich nicht selbst



Bilder: Wikipedia

Cube mapping

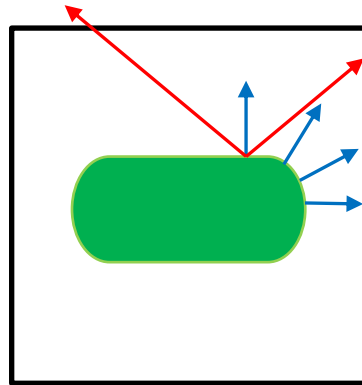
- Mittelpunkt
= Kamera
- Rendere 6 Seiten
- Ergebnisse als
Texturen
verwenden
- Dynamisch sehr
rechenintensiv



Bilder: Wikipedia

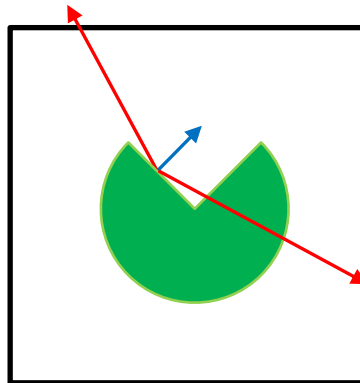
Vom Würfel zu anderen Formen

- Objekt wird vom Würfel umschlossen
- Texturkoordinaten nicht am Objekt gespeichert
- Normale interpolieren
- Für jeden Pixel
 - Vektor zur Kamera an Normale reflektieren
 - In Reflexionsrichtung Schnitt mit Würfel bestimmen
 - Texturinformationen vom Schnittpunkt



Problem

- Nicht-konvexe Objekte sind schwierig
- Spiegelt sich nicht selbst



Light / Shadow maps

- Licht/Schatten auf Textur vorberechnen
 - Spart Berechnung bei statischen Lichtquellen
 - Änderung der Umgebung nicht berücksichtigt

Bilder: epicgames.com



Computergrafik - Prof. Dr. Tobias Lenz

219

Light maps

- Realität: Sehr viele lokale Lichtquellen
 - Rechenleistung reicht nicht aus
- Illusion: Einmalig vorberechnete Lichttextur wird mit normaler Textur kombiniert
 - Niedrige Auflösung reicht



Computergrafik - Prof. Dr. Tobias Lenz

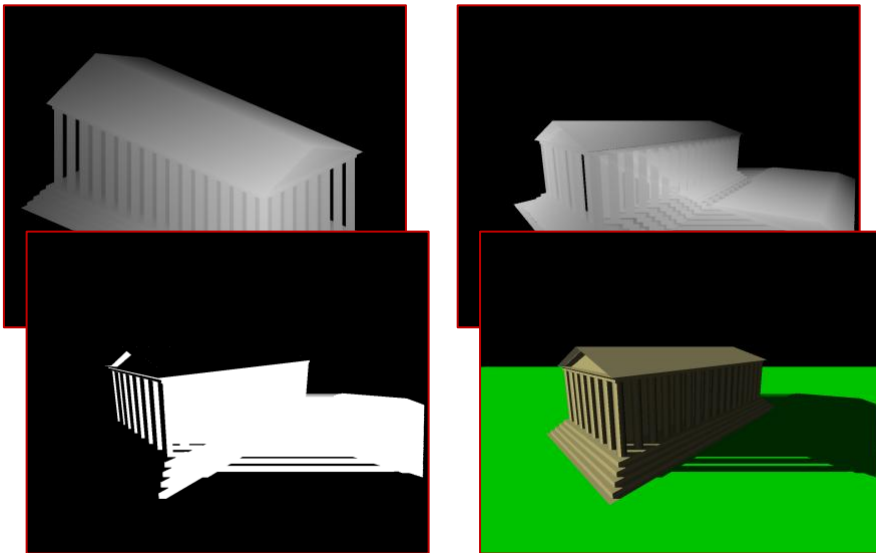
220

Bilder: flipcode.com

Shadow map: Verfahren

- Szene aus Sicht des Lichts berechnen
 - Z-buffer reicht, Farbe/Beleuchtung egal
 - Lichtkoordinatensystem
- Szene aus Sicht der Kamera berechnen
 - Pro Pixel Abstand d zum Licht berechnen
 - Transformation in Lichtkoordinatensystem
 - Mit Tiefe t im z-buffer vergleichen
 - Wenn $d < t$ dann Licht, sonst Schatten

Shadow map: Beispiel



Shadow map: Probleme

- Numerische Genauigkeit unzureichend
 - Lichtabstand = Tiefe im z-buffer
 - „kleiner gleich“ bei floats nicht stabil
 - Ansatz: Lichtabstand künstlich verkleinern
- Hohe Auflösung für visuell gute Ergebnisse
- Genauigkeit des z-buffers nicht ausreichend bei großen Schatten
 - Flackern bei dünnen Strukturen

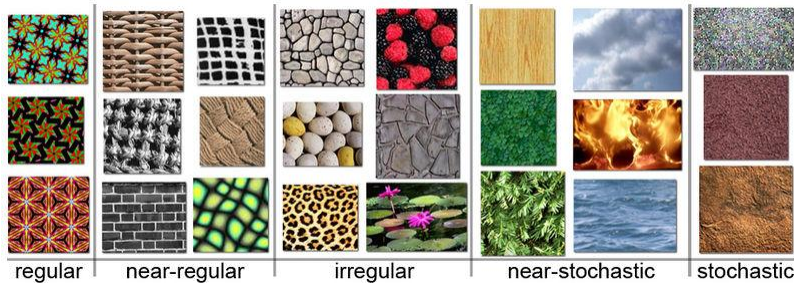
Übersicht Texturverfahren

- Immer gleich: Aufwändige Berechnungen durch (vorgefertigte) Bilder ersetzen

Name	Verwendung
Billboarding	Ganze Objekte durch Texturen ersetzen
Bump mapping	Oberflächennormalen manipulieren
Normal mapping	Oberflächennormalen ersetzen
Displacement mapping	Geometrie manipulieren / verfeinern
Environment / Reflection mapping	Spiegelnde Oberflächen simulieren
Light / Shadow maps	Licht / Schatten vorberechnen

Prozedurale Textur

- Textursynthese = „Künstliches“ Erzeugen einer Textur
- Grundlage: Beobachtung verschiedener Muster in der Natur



Textursynthese: Varianten

- Textur zum Programmstart per Funktion erzeugen
 - Textur muss nicht gespeichert / geladen werden
 - Kompaktes Programm
- Textur nur für angefragte Koordinaten dynamisch erzeugen
 - Speicherverbrauch stark reduziert
 - Auflösungsunabhängig

Prozedurale Textur: Vergleich

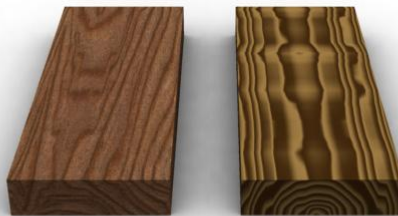
Bild als Textur

- Hoher Speicherverbrauch
 - MIP maps, Anisotropie, ...
- Feste (Maximal-)Größe
- Änderungen in Grafikprogramm
- Aufwändig zu animieren
- Realistische Wirkung

Prozedurale Textur

- Kein Speicherbedarf
- Auflösungsunabhängig
- Alle Parameter schnell änderbar
- Einfach zu animieren
- Wirkt häufig künstlich, Realitätsnähe schwierig

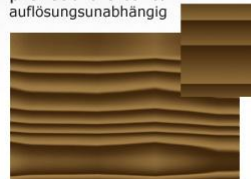
Prozedurale Textur: Vergleich



bitmap textur
auflösungsabhängig



prozedurale textur
auflösungsunabhängig



Texturen als „Datenablage“

- Texturen bisher hatten immer irgendwie mit Bildern im weiteren Sinne zu tun
- Rein mathematisch können Texturen auch als beliebige 1/2/3d-Funktion betrachtet werden
 - Texelfarbe gibt Funktionswert wieder
 - Werte zwischen Texeln werden interpoliert
- Häufig schneller als Funktion auszuwerten
 - z.B. Rauschen, Zufallszahlen

Texturen in Java

- Zum VAO ein VBO mit uv-Koordinaten pro Ecke hinzufügen
- Bilddatei als Textur laden → gibt textureID
- `glBindTexture(GL_TEXTURE_2D, textureID)`
- Im Vertex Shader uv-Koordinaten annehmen und direkt an Fragment Shader durchreichen
- Fragment Shader

uniform **sampler2D** smplr;

vec4 texel = texture(smplr, uvKoordinaten);